

Verifacts Technical Specification

Version: 1.0
Date: November 22, 2025
Status: Draft

1. Executive Summary

Veritas is a low-latency source verification system designed to verify news content via browser extensions and mobile share intents. The system utilizes a "Hub-and-Spoke" architecture where a central API Gateway orchestrates parallel requests to three distinct analysis services ("Agents").

Key Constraints:

- **Latency:** Total request processing time must be < 3,000ms.
- **Throughput:** System must handle high concurrency via aggressive caching.
- **MVP Scope:** Limited to lookup-based source verification and API-based fact-checking. Real-time web crawling is out of scope for V1.

New in v1.1: Support for **Granular Verification** (highlighting specific sentences) alongside full-page verification.

2. System Architecture

The architecture follows a micro-modular design patterns centered around a non-blocking API Gateway.

2.1 Core Components

Component	Technology Stack	Responsibility
Client: Browser Ext	React, Vite, Manifest V3	DOM injection, URL capture, Context Menu handling.
Client: Mobile App	React Native / Flutter	Share Intent handling, Text Selection Intent .
API Gateway	Python (FastAPI)	Request validation, Caching (Redis), Orchestration.
Agent 1 (Source)	Python, PostgreSQL	Static domain lookup

		(Bias/Reliability).
Agent 2 (Claims)	Python, OpenAI API	HTML parsing OR Raw Text Analysis .
Agent 3 (Verify)	Python, Google APIs	Fact-check database querying.
Cache Layer	Redis	Storing JSON reports keyed by URL hash (or Text hash).

3. Component Specifications

3.1 Frontend Clients

A. Browser Extension (Chrome/Firefox)

- **Isolation:** Must use **Shadow DOM** to inject the sidebar UI.
- **Context Menu Integration (New):**
 - **Permission:** Add contextMenus to manifest.json.
 - **Event:** Listen for chrome.contextMenus.onClicked.
 - **Logic:** When user highlights text > Right Click > "Verify with Veritas", capture info.selectionText and send to Backend with the page URL.

B. Mobile Application (iOS/Android)

The mobile app functions primarily as a **Share Target** and **Text Processor**.

- **Android Implementation:**
 - **Full Page:** Listen for ACTION_SEND (URL sharing).
 - **Text Selection (New):** Register ACTION_PROCESS_TEXT intent. This adds "Veritas" to the floating toolbar when a user highlights text in *any* app (Chrome, Twitter, WhatsApp).
- **iOS Implementation:**
 - **Share Extension:** Configure NSExtensionActivationSupportsText to YES.
 - **Logic:** Check NSExtensionItem attachments. If kUTTypeURL exists, verify URL. If kUTTypeText exists, verify the raw string.

3.2 Backend API (The Orchestrator)

- **Framework:** FastAPI (Python).
- **Routing Logic:**
 - If selection is present: Skip Agent 1 (Source Analysis) *unless* a URL is also provided. Prioritize Agent 3 (Fact Check) using the specific selected text.

- If selection is empty: Run full pipeline (Source + Extraction + Verify).

3.3 Service Layer (The Agents)

Agent 1: Source Analyzer (Lookup Service)

- **Mechanism:** SQL Query.
- **Logic:** SELECT bias, reliability FROM sources WHERE domain = {normalized_domain}.

Agent 2: Claim Extractor (NLP Service)

- **Library:** newspaper3k (for HTML body extraction).
- **Model:** gpt-4o-mini.
- **Dual Mode:**
 1. **Full Mode:** Extract claims from full HTML.
 2. **Passthrough Mode (New):** If selection is provided, sanitize the text and pass it directly to Agent 3 as the "Claim" (bypassing extraction), or use LLM to simply "atomize" the selection into checkable facts.

Agent 3: Fact-Checker (Verification Service)

- **Primary API:** Google Fact Check Tools API.
- **Logic:** Query API with claims.
- **Fallback:** Return "Unverified" if no matches found.

3.4 Data Layer

- **Redis (Caching):**
 - **Key Strategy:**
 - Full Page: SHA256(normalized_url)
 - Selection: SHA256(selection_text)
- **PostgreSQL:** Stores user usage logs.

4. Data Flow (Text Selection)

1. **Trigger:** User highlights "Vaccines cause magnetism" -> Right Click -> Verify.
2. **Client:** Sends POST /verify with payload { url: "...", selection: "Vaccines cause magnetism" }.
3. **Check Cache:** Backend hashes the *selection text*.
 - *Hit:* Return cached verification.
4. **Orchestrate (Miss):**
 - *Agent 1:* Checks domain (if URL provided).
 - *Agent 2:* (Fast Path) Cleans up selection text (removes weird chars).
 - *Agent 3:* Queries Google Fact Check API with the selection.
5. **Response:** Returns specific verification for that sentence.

5. Implementation Phases (MVP)

Phase 1: Skeleton & Mocking (Week 1)

- **Objective:** End-to-end connectivity.
- **Deliverables:** FastAPI server, Redis, Browser Ext "Hello World".

Phase 2: Source Intelligence (Week 2)

- **Objective:** Implement Agent 1.
- **Deliverables:** Postgres schema, CSV ingestion.

Phase 3: The AI Pipeline (Week 3)

- **Objective:** Implement Agents 2 & 3 + **Text Selection**.
- **Deliverables:**
 - OpenAI/Google API integration.
 - **Context Menu** listener in Chrome Extension.
 - **Android Intent** for text processing.

Phase 4: Optimization & Release (Week 4)

- **Objective:** Production readiness.
- **Deliverables:** Redis caching, Production keys, Store submission.

6. API Specification (Draft)

POST /api/v1/verify

Request:

```
{
  "url": "[https://example.com/article-slug](https://example.com/article-slug)",
  "selection": "Optional string. If provided, verifying this specific text.",
  "force_refresh": false
}
```

Response:

```
{
  "status": "success",
  "mode": "granular",
  "data": {
    "source": {
      "name": "Example News",
      "bias": "left-center"
    },
    "claims": [
      {
```

```
    "claim": "Vaccines cause magnetism",  
    "status": "debunked",  
    "corroboration_url": "[https://snopes.com/](https://snopes.com/)..."  
  }  
]  
}  
}
```